



温州肯恩大学
WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

Intro to Web Data Collection & GitHub Literacy

Dawei Chen

Week 8-2, 16 April 2026

Remaining Semester

The latest version of the syllabus is available in Canvas.

The remaining sessions will focus on **two** core business applications of Python.

Data Collection

Week	Date	Topic	Note
w8	4/14	Research Workshop II	Hypothesis, measurement
	4/16	Intro to Web Data Collection & GitHub Literacy	TODAY
w9	4/21	API-Based Data Collection	HTTP request/response, GitHub API demo
	4/23	Working with JSON & Semi-Structured Data	JSON parsing, flattening, export to CSV
w10	4/28	HTML Scraping & Unstructured Data	HTML structure, requests + BeautifulSoup, text scraping
	4/30	Research Workshop III (<i>tentative</i>)	No lecture ; data collection troubleshooting

Remaining Semester

The latest version of the syllabus is available in Canvas.

The remaining sessions will focus on **two** core business applications of Python.

Data Analysis

Week	Date	Topic	Note
w11	5/07	Structured Data Cleaning	Pandas, missing values, duplicates, types
w12	5/12	Unstructured Data Processing	Text cleaning, regex, variable construction
	5/14	Data Integration & Research-Ready Dataset	Merge/join, feature engineering, validation
w13	5/19	Exploratory Data Analysis	Descriptive stats, correlations, Pandas
	5/21	Data Visualization	Principles, Matplotlib, Seaborn
w14	5/26	Regression: Theory	Coefficients, p-values, model specification
	5/28	Regression: Application	statsmodels OLS, diagnostics, results

Learning Objectives

By the end of this session, you should be able to:

- Understand the big picture of web data collection
 - What methods exist
 - When to use each
 - What makes some sources easy or hard to scrape
- Distinguish between structured, semi-structured, and unstructured web data
- Navigate GitHub: find, evaluate, and clone relevant open-source projects
- Run an existing Jupyter notebook from GitHub in VS Code

Introduction to Web Data Collection

What data lives on the web, how do you get it, and what makes it easy or hard?

Why Collect Data from the Web?



The web is the largest source of real-world behavioral data.

- User activity on platforms (GitHub, Stack Overflow, Reddit, etc.)
- Market data, product listings, reviews
- Government and public datasets
- News, social media, job postings

Studies from your reading list collect data from:

- **GitHub** — Copilot & open-source activity
- **Stack Overflow** — Q&A patterns after ChatGPT
- **Freelancing platforms** — AI impact on labor
- **Pixiv** — AI & human creators

**Your Research
Projects?**

You will collect similar data for your final project using the techniques we learn in the next 4 sessions.

Types of Web Data

Structured

Highly organized, fixed format
Stored in rows and columns
Easy to store, query, analyze

E.g. excel spreadsheet, SQL database, CSV file

“ideal case”, clean, easy

Semi-Structured

No fixed table structure
Still contains labels/tags
Flexible but partially organized

E.g. JSON (APIs return), XML, HTML (webpages)

Most web data is semi-structured

```
<> JSON
{
  "name": "Alice",
  "age": 25,
  "city": "New York"
}
```

```
<> XML
<product>
  <name>iPhone</name>
  <price>999</price>
</product>
```

Unstructured

No predefined format
No clear rows, columns, tags
Difficult to analyze directly

E.g. text (reviews, posts), images, videos

High value but harder to process
This is where AI becomes useful

How Websites Work — The Basics

Frontend

vs.

Backend

HTML

CSS

JavaScript

Server

Database

API



This is the foundation. Every scraping method works by intercepting or replicating some part of this cycle.

Key Concepts

- **HTTP** (hypertext transfer protocol) — the protocol (language) browsers use to talk to servers
- **HTML** (hypertext markup language) — the structure of a webpage (what BeautifulSoup reads)
- **CSS** (cascading style sheets) — the styling (colors, fonts, layout)
- **JavaScript** — the code that makes pages interactive and loads dynamic content

Two Doors to Web Data

API: Application Programming Interface

❖ API — The Front Door

The website gives you data through an official interface

- You send a structured request
- You get structured data back (usually JSON)
- Clean, reliable, intended for programmatic use
- Often requires authentication (API key/token)
- Has rate limits (requests per hour)

HTML Scraping — Reading the Page

You parse what the browser displays

- Download the HTML of a webpage
- Extract data from tags and elements
- Data is mixed with layout, ads, navigation
- More fragile (breaks if site redesigns)
- Some sites try to block scrapers

Some sites offer both. Some offer only one. Some try to block both.

The Easy End — APIs and Static Pages

✓ These are beginner-friendly and where you should start.

Well-Documented APIs

- [GitHub API](#) — repositories, commits, issues
- [Reddit API](#) — posts, comments, subreddits
- Government open data portals
- Many platforms provide official APIs

Static HTML Pages

- Content visible in page source code
- Wikipedia, many news sites, blogs
- Government reports, academic pages
- Public CSV/Excel file downloads

How to check:

- ✓ Right-click on a webpage → **“View Page Source”**
- ✓ If you can find the data in the HTML, it’s static and easy to scrape.

Static vs. Dynamic Websites

Why this matters: your scraping approach depends entirely on whether the site is static or dynamic.

Static

Content is in the HTML source code

- What you see in “**View Page Source**” is what you get
- BeautifulSoup + requests can handle these
- Page loads once with all content

Examples: Wikipedia, many news sites, government pages, blogs



Dynamic

Content is loaded by JavaScript after the page loads

- Page source is mostly empty or contains JavaScript code
- BeautifulSoup alone cannot see this content
- Infinite scroll, “load more” buttons, content that appears after clicking

Examples: social media feeds, e-commerce product pages, SPAs

The Middle Ground — Dynamic Content

When a website loads content dynamically (via JavaScript), you have two main approaches:

Approach 1: Find the Hidden API

“JS逆向” — reverse-engineering the data requests

- Open browser Developer Tools (F12)
- Go to Network tab → filter by XHR/Fetch
- Find the actual data request the browser makes
- Replicate that request in Python

Pro: Fast, returns clean data (usually JSON)

Con: Requires detective work; may need cookies/headers

Approach 2: Browser Automation

“模拟手动点击” — simulating manual clicks

- Tools like Selenium or Playwright
- Launches a real browser programmatically
- Can click buttons, scroll, wait for content
- Sees the fully rendered page

Pro: Works on almost any website

Con: Slow; resource-heavy; easier to detect

*In this course, we focus on **APIs** and **static scraping**.*

Anti-Scraping Mechanisms (反爬)

Why do websites block scrapers?

Server load protection | Commercial data protection | User privacy

IP Blocking & Rate Limiting

Too many requests from one IP → blocked

CAPTCHAs

“Prove you’re human” tests

Login Walls

Content only available after authentication

Cookies & Session Tracking

Track visitor behavior to detect bots

User-Agent Detection

Check if the visitor is a real browser or a script

JavaScript Challenges

Cloudflare-style protection that blocks non-browser requests

- It’s an arms race between scrapers and websites.
- You don’t need to win — just understand the landscape so you can choose feasible data sources.

What's Easy vs. Hard to Scrape

Easy	Medium	Hard	Very Hard
Open APIs with docs	Hidden APIs (found via browser dev tools)	JS-rendered SPAs	CAPTCHA-protected
Government data portals	Paginated content	Cloudflare protection	Active bot detection
Static HTML pages	Sites requiring simple authentication/cookies	Mobile app data	Paid content walls
Wikipedia		Strict rate limits (e.g., social media)	Personal/private data (legal & ethical issues)
Public CSV downloads			

For your research project:

- Start with the easy end. Look for sites with APIs or static HTML.
- If your ideal data source is on the hard end, consider whether an easier alternative could answer your research question just as well.

Python Tools for Web Data Collection

Method	Python Tools	Best For	Difficulty
API Calls	requests, httpx	Sites with official APIs (GitHub, Reddit, Twitter)	★
HTML Parsing	requests + BeautifulSoup	Static pages (Wikipedia, news, blogs)	★★
Hidden API Discovery	requests + browser DevTools	Dynamic sites with discoverable XHR calls	★★★
Browser Automation	Selenium, Playwright	JS-heavy sites, click-through pages	★★★★
Scraping Frameworks	Scrapy	Large-scale crawling, complex pipelines	★★★★

In this course:

- We focus on the first two rows (API calls + HTML parsing).
- These cover most research data needs.
- AI can generate the code — your job is to understand which approach fits your data source.

The AI Advantage in Data Collection

You don't need to master all scraping techniques yourself.

- AI tools can generate scraping code for most scenarios.
- Your job is to apply the 4D framework:

Delegation

Understand what kind of data source you're dealing with before asking AI to write code

Description

Describe the target site, the data fields you need, the output format, and any constraints (pagination, auth)

Discernment

Check if the generated code actually works — does it return the right data? Handle errors? Respect rate limits?

Diligence

Ensure ethical and legal compliance — robots.txt, ToS, reasonable request rates

Ethical & Legal Awareness

robots.txt

A file on every website that tells scrapers what they can and cannot access. Always check it first.

Example: GitHub's robots.txt

Terms of Service

Many sites explicitly prohibit automated data collection. Read the ToS before scraping commercially or at scale.

Rate Limiting

Be a polite scraper: add delays between requests (e.g., 1–2 seconds). Don't overwhelm servers.

Personal Data & Privacy

Be extremely cautious with data that identifies individuals. GDPR and other regulations apply.

Rule of thumb: If data is publicly accessible, you collect it at a reasonable pace, and you don't bypass access controls, you're generally in safe territory. When in doubt, look for an official API first.

Before You Start Coding: Assess Your Data Source

Before writing any code (or asking AI to write it), answer these questions:

- | | | |
|----------|---------------------------------------|--|
| 1 | What data do I need? | Which fields, how many observations, what time period? |
| 2 | Where does it live? | Which website or platform? Is it one page or many? |
| 3 | Is there an API? | Check the site's developer documentation. |
| 4 | Is the site static or dynamic? | Right-click → View Page Source. Can you find the data? |
| 5 | What obstacles might I face? | Login required? Rate limits? Anti-scraping protection? |
| 6 | Is it ethical and legal? | Check robots.txt and ToS. Is the data public? |

GitHub Literacy

Finding, evaluating, and reusing open-source code for your projects

What Is GitHub?



GitHub is like Google Drive for code — but it also keeps the history of every change.

For this course, GitHub is useful in two ways:

Save & organize your own work

- Version control for your project code
- No more “final_v2_real_final.ipynb”
- Backup and access from anywhere

Find & reuse others' code

- Open-source scraping scripts
- Data collection templates
- Starter code for your research

***Quick check:** How many of you created a GitHub account after the Week 7-2 exercise?*

Anatomy of a GitHub Repository



README.md

The “front page” — explains what the project does, how to use it



Stars

How many people “liked” it — a rough quality signal



Issues

Bug reports and feature requests — shows community activity



Code files

The actual scripts, notebooks (.py, .ipynb), organized in folders



Forks

How many people copied it to modify — shows active reuse



Last updated

When was it last changed? Stale repos may have outdated code

We'll look at a real repository together in the live demo.

How to Search GitHub for Useful Projects

Search strategies for finding scraping code or data collection tools:

- **Use specific keywords:** “scraper” + platform name (e.g., “github scraper”, “reddit scraper python”)
- **Filter by language:** Use GitHub’s language filter to show only Python projects
- **Sort by stars:** Higher stars generally means better quality and documentation
- **Check “Topics”:** Repos are tagged with topics like “web-scraping”, “data-collection”, “api”
- **Look for Jupyter notebooks:** Search with “.ipynb” in the query — easier to understand and run
- **Browse curated lists:** Search for “awesome-web-scraping” or “awesome-python” for community-maintained lists

How to Evaluate a Repository

Not every project on GitHub is worth using. Ask these questions:

Question	✓ Good Sign	✗ Warning Sign
Is the README clear?	Explains purpose, how to install, how to run	Empty or vague README
Is it recently maintained?	Updated within the last year	Last commit 3+ years ago
Does it have stars/forks?	50+ stars suggests community trust	0 stars, no activity
Are dependencies reasonable?	Uses common libraries (requests, pandas)	Requires obscure or paid tools
Does it match your needs?	Collects data from your target site	Similar but different platform

Fork, Clone, and Open in VS Code

- 1 Find** Search GitHub for a project relevant to your research
- 2 Fork** Click “Fork” to create your own copy on GitHub
- 3 Clone** Download the copy to your computer
(green “Code” button → copy URL → git clone)
- 4 Open** Open the folder in VS Code
(File → Open Folder)
- 5 Run** Install dependencies (!pip install ...)
Open the notebook and run cells

We’ll walk through this together in the live demo →

Live Demo

Let's do this together in VS Code

1. Find a GitHub repo with a Python scraping example
2. Read the README and evaluate it
3. Fork and clone it to our computer
4. Open in VS Code
5. Install dependencies and run the notebook

Hands-On: Your Turn

Task (remaining class time):

- Search GitHub for an open-source project related to your research topic or data source
- Evaluate the repository: Is the README clear? Is it maintained? Does it match your needs?
- Fork and clone the project to your computer
- Open it in VS Code and try to run it
- If you get stuck, that's normal — ask AI, try again

Next session (4/21)

→ API-Based Data Collection — we'll use the GitHub API to collect real data for our demo dataset.

If you don't have a GitHub account yet, create one now: github.com

Q&A

CBPM B214

dchen@kean.edu

Tue & Thu 11:15 – 12:30 | Wed 14:00 – 16:30